

SOCKET I/O MODES AND OTHERS

Byeong-hee Roh

Dept. of Software and Computer Engineering

Ajou University



변화와 융합의 중심

MMcN

멀티미디어 융합 연구실

Mobile Multimedia convergene Network Lab.

<http://mmcn.ajou.ac.kr>



Contents

- ❖ Socket I/O Modes
- ❖ Nonblocking I/O Mode
- ❖ Socket Timeout
- ❖ Other Socket Functions



SOCKET I/O MODES



Socket I/O Mode

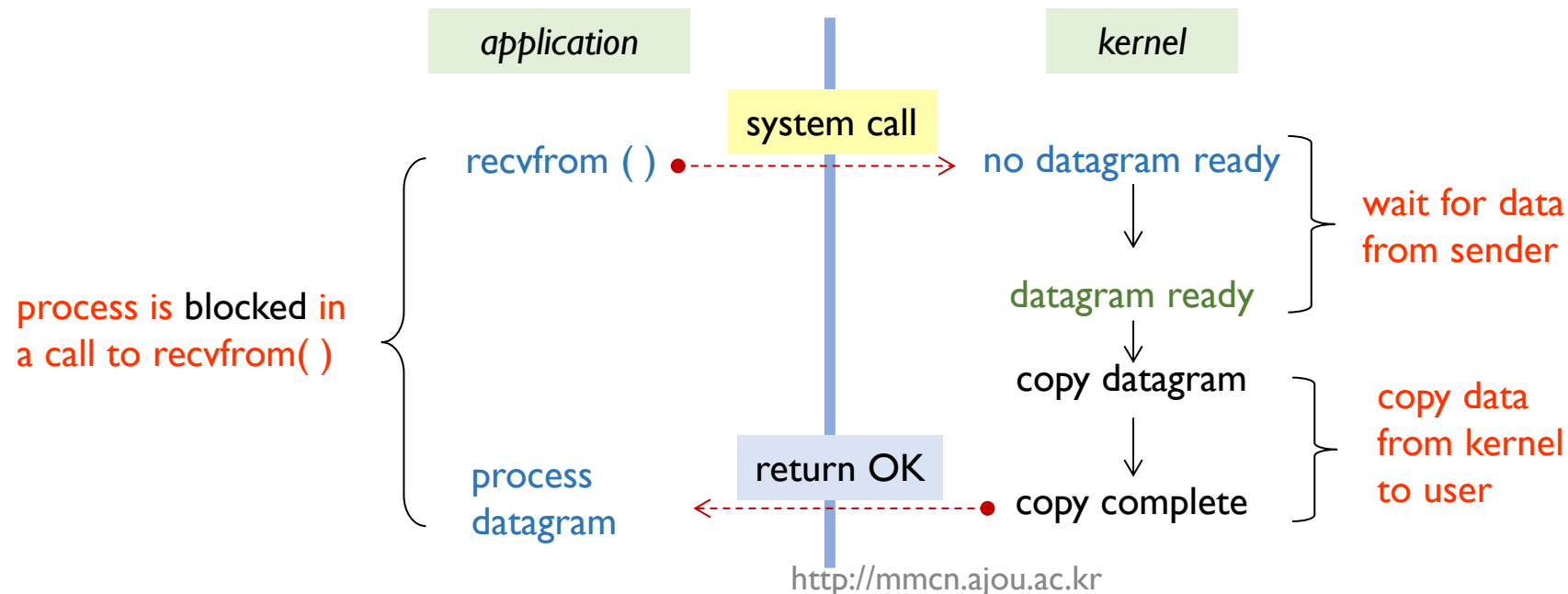
❖ Socket I/O Modes

- Blocking I/O
- Nonblocking I/O
- I/O multiplexing
- Signal driven I/O
- Asynchronous I/O

Blocking I/O

❖ Blocking I/O

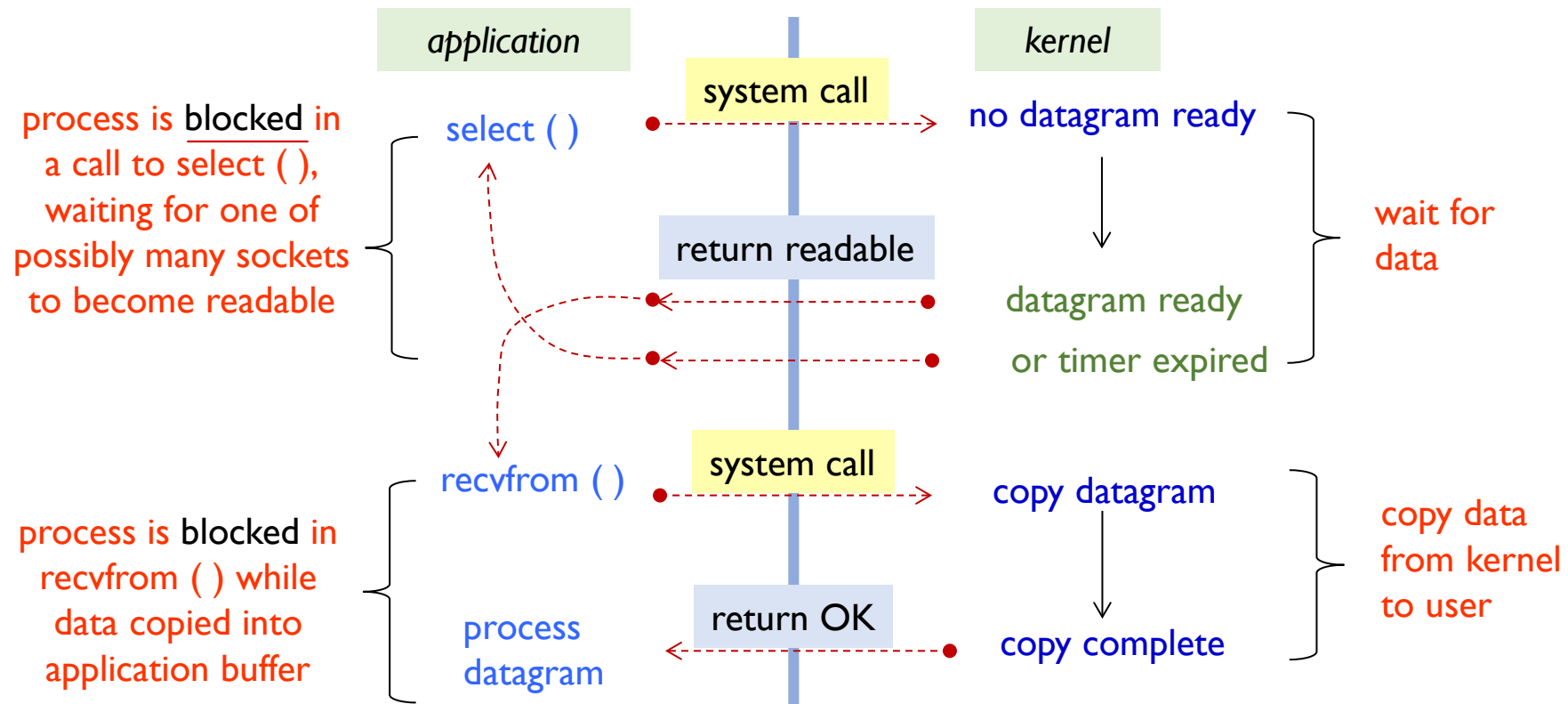
- when a system call is called on a socket,
 - the called process waits until the kernel completes the request
- related functions
 - `listen()`, `connect()`, `accept()`, `recv()`, `send()`, `read()`, `write()`, `recvfrom()`, `sendto()`, `close()`, etc.



I/O Multiplexing

❖ I/O multiplexing

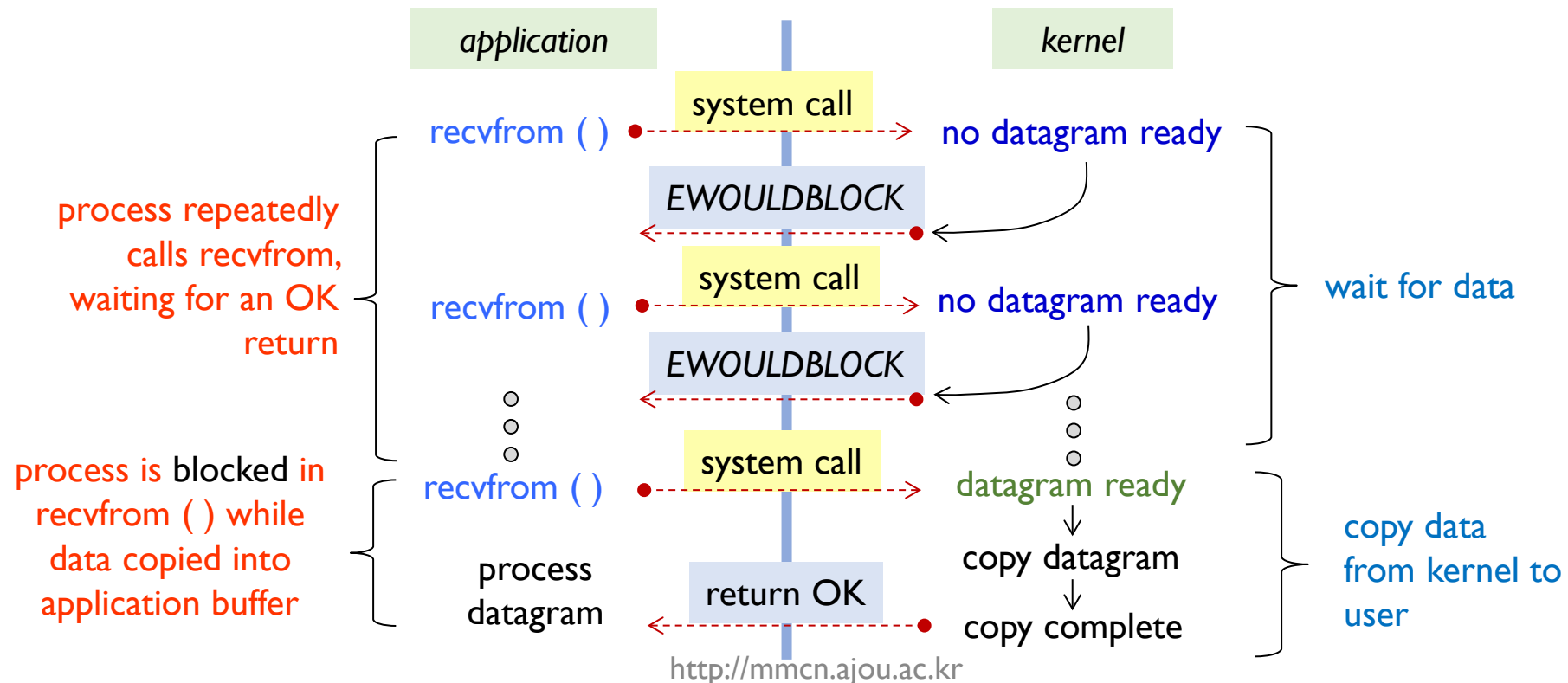
- notifies if one or more I/O conditions are ready
- related functions : `select( )`, `poll( )`, etc.



Nonblocking I/O

❖ Nonblocking I/O

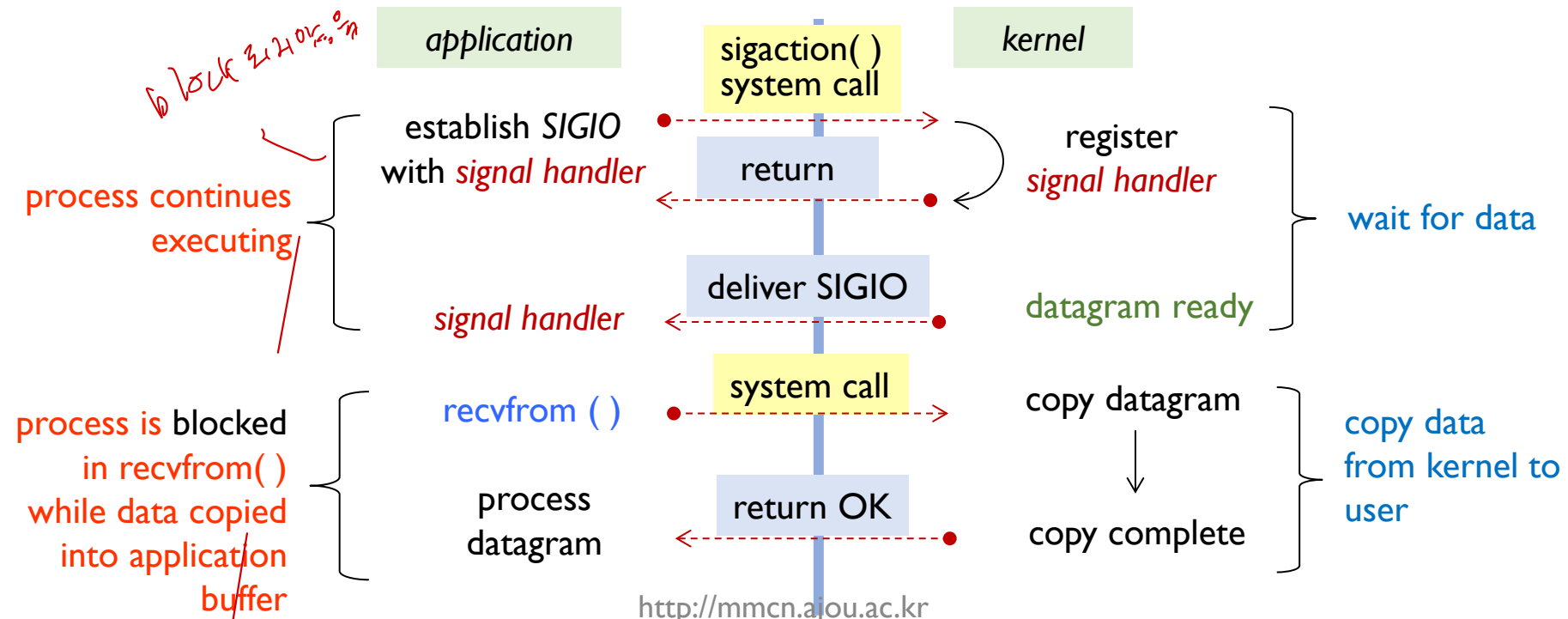
- when a system call is called on a socket
 - the kernel immediately returns though the request can not be completed
- if the request didn't be completed, the system call can be tried again.



Signal Driven I/O

❖ Signal Driven I/O

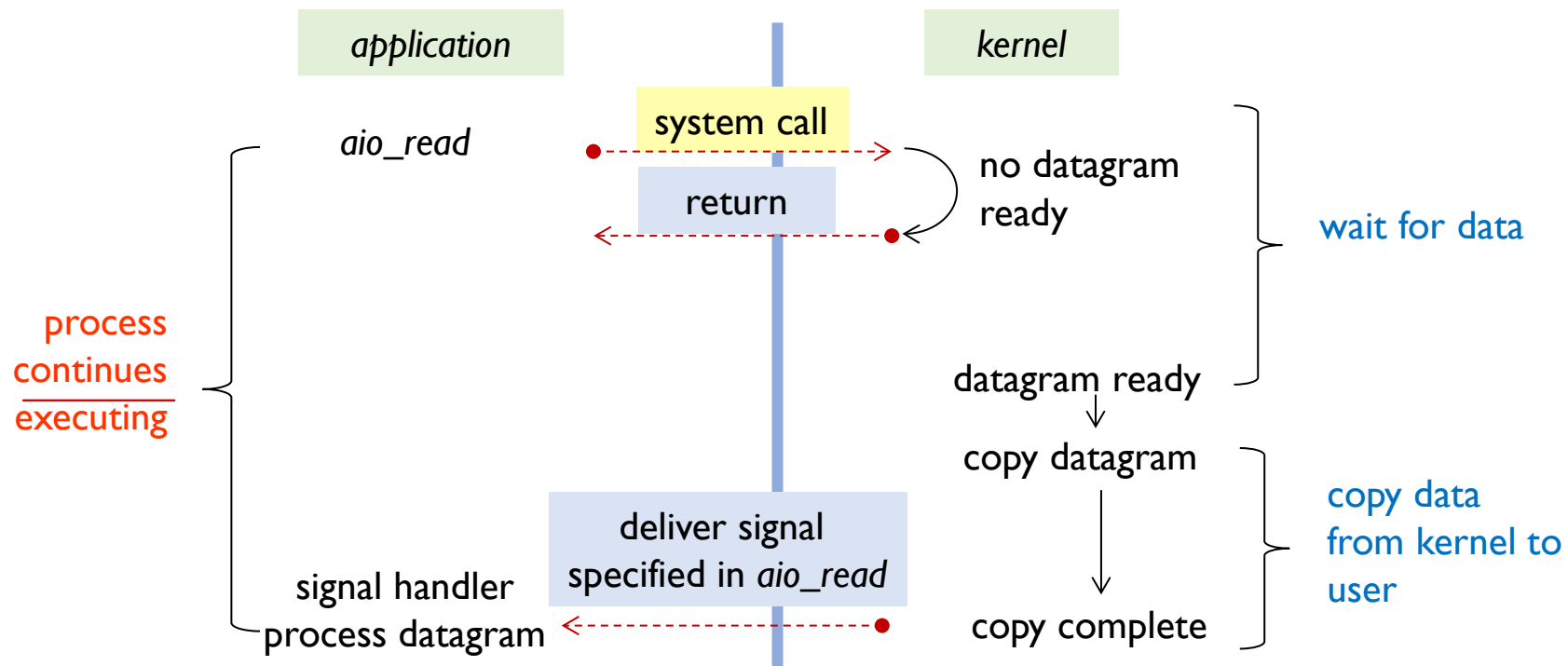
- telling the kernel to notify us with the *SIGIO* signal when the descriptor is ready
 - enable the socket for signal driven I/O
 - install a signal handler using the *sigaction* system call



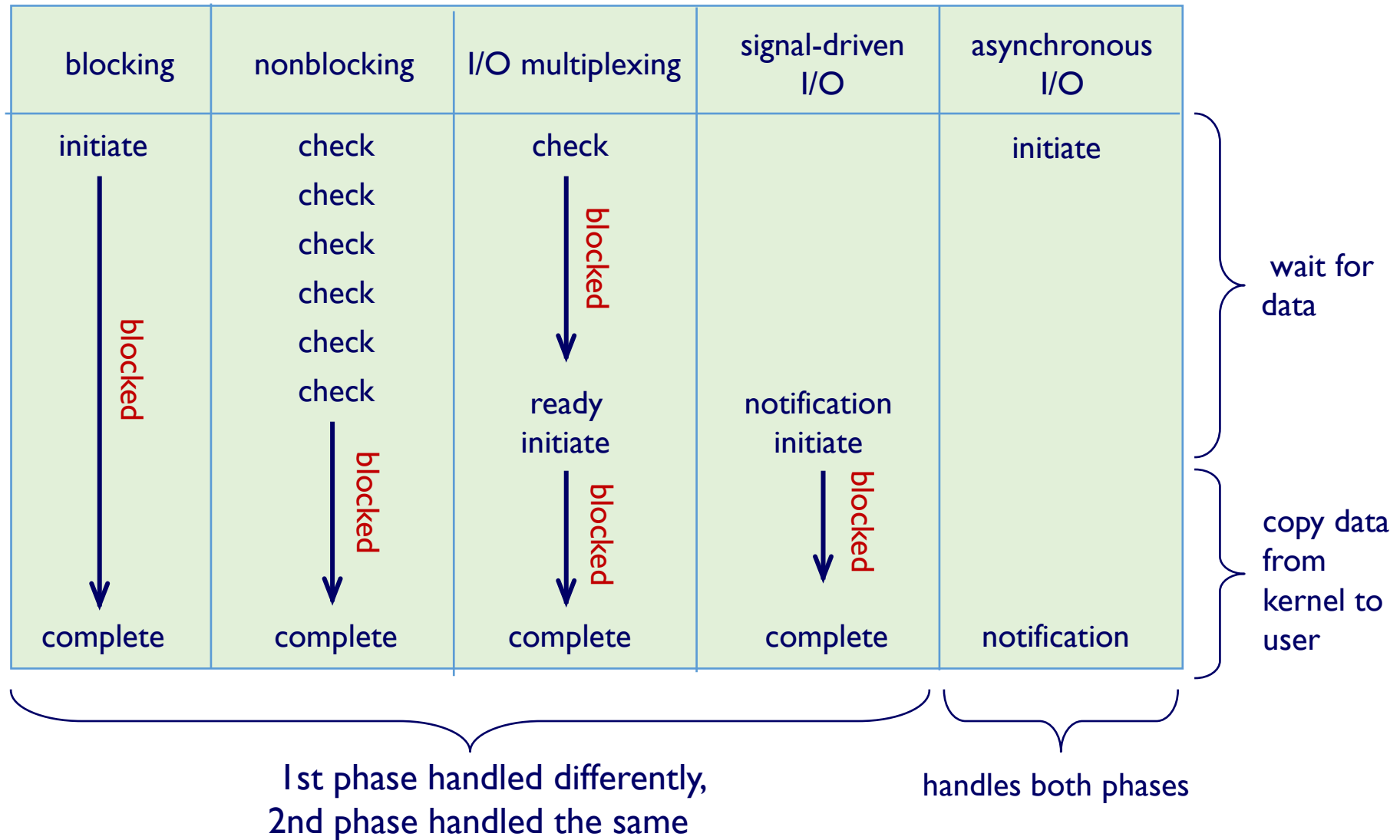
Asynchronous I/O

❖ Asynchronous I/O

- telling the kernel to start the operation and to notify us when the entire operation is complete.



Comparison of I/O Modes



Comparison of I/O Modes

❖ Socket I/O Modes

- Blocking I/O and I/O multiplexing
 - Processes are *blocked* until some events to return to the calling function occur.
- Nonblocking I/O
 - There is no way to know when the events on the socket happen.
- Signal driven I/O (or Asynchronous I/O)
 - It can inform the process that something happens on the socket
 - POSIX defines Synchronous I/O and Asynchronous I/O as follows
 - A **synchronous I/O operation** causes the requesting process to be blocked until that I/O operation completes
 - blocking, I/O multiplexing, nonblocking, and signal driven I/O modes
 - An **asynchronous I/O operation** does not cause the requesting process to be blocked
 - only the asynchronous I/O mode

Ref W. Richard Stevens, UNIX Network Programming, The Sockets Networking API, Vol. I, 3rd Ed.



NONBLOCKING I/O MODE

Nonblocking I/O Mode

❖ ioctlsocket()

- controls the I/O mode of a socket

```
int ioctlsocket ( SOCKET      s,  
                 long        cmd,  
                 u_long FAR*  argp );
```

■ arguments

- **s** : the descriptor identifying a socket.
- **cmd** : the command to perform on the socket **s**.
 - **FIONBIO** : nonzero to enable the **nonblocking** mode of socket **s**
 - **FIONREAD** : use to determine the amount of data pending in the network's input buffer that can be read from socket **s**.
 - **SIOCATMARK** : use to determine whether or not all out-of-band data has been read.
- **argp** : the pointer to a parameter for **cmd**

Nonblocking I/O Example for Winsock

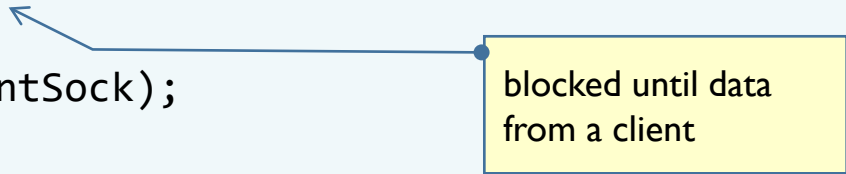
❖ Example : Blocking I/O case

```
...
/* Create socket for incoming connections */
servSock = socket (PF_INET, SOCK_STREAM, IPPROTO_TCP));
...
/* Bind to the local address */
bind (servSock, (struct sockaddr *) &echoServAddr, sizeof(echoServAddr));
listen (servSock, MAXPENDING) ;

for ( ; ; ) /* Run forever */ {

    /* Wait for a client to connect */
    clntSock = accept (servSock, (struct sockaddr *) &echoClntAddr, &clntLen)) ;

    HandleTCPClient(clntSock);
}
```



A blue arrow points from a yellow callout box to the `accept` function call in the code. The callout box contains the text "blocked until data from a client".

Nonblocking I/O Example for Winsock

❖ Example : NonBlocking I/O case

```
...
servSock = socket (PF_INET, SOCK_STREAM, IPPROTO_TCP));

int on=1;
ioctlsocket (servSock, FIONBIO, &on);

bind (servSock, (struct sockaddr *) &echoServAddr, sizeof(echoServAddr));
listen (servSock, MAXPENDING) ;
while ( 1 ) {
    clntSock = accept (servSock, (struct sockaddr *) &echoClntAddr, &clntLen)) ;
    if ( clntSock == INVALID_SOCKET ) { printf("."); continue; }
    while ( 1 ) {
        retval = recv (clntSock, buf, BUFSIZE, 0);
        if(retval == SOCKET_ERROR) { printf("*"); continue; }
        ...
        retval = send (client_sock, buf, BUFSIZE, 0);
    }
    ...
}
```

↳ ioctlsocket(clntSock, FIONBIO, &on);

<http://mmcn.ajou.ac.kr>

Nonblocking I/O Example for Winsock

Blocking I/O Mode

```
C:\NetSW>C:\echoSvr.exe

[TCP Server] Client accepted : IP addr=127.0.0.1, port=58184
[TCP/127.0.0.1:58184] Hello World !!
[TCP/127.0.0.1:58184] I am Client, You are Server, Right ?
[TCP/127.0.0.1:58184] This Network Software Design Class
[TCP/127.0.0.1:58184] 네트워크소프트웨어설계 과목입니다.
[TCP 서버] 클라이언트 종료: IP 주소=127.0.0.1, 포트 번호=58184
```

Nonblocking I/O Mode

```
C:\NetSW\04-AsyncSocket\NonblockSvr\Debug\NonblockSvr.exe
# Nonblocking Server:
# Network SW Design Course by Ajou University

.....
[TCP Nonblocking Server] Client connected (127.0.0.1:60718)
*****
[TCP Nonblocking Server] Message from client (127.0.0.1:60718): echoed: hello
*****
[TCP Nonblocking Server] Message from client (127.0.0.1:60718): echoed: Hi
*****
[TCP Nonblocking Server] Message from client (127.0.0.1:60718): echoed: message
*****
[TCP Nonblocking Server] connection terminated: client (127.0.0.1:60718)d
.....
[TCP Nonblocking Server] Client connected (127.0.0.1:60720)
*****
[TCP Nonblocking Server] Message from client (127.0.0.1:60720): echoed: second
*****
[TCP Nonblocking Server] Message from client (127.0.0.1:60720): echoed: message
*****
[TCP Nonblocking Server] Message from client (127.0.0.1:60720): echoed: kkkkkk
*****
[TCP Nonblocking Server] connection terminated: client (127.0.0.1:60720)d
.....
```

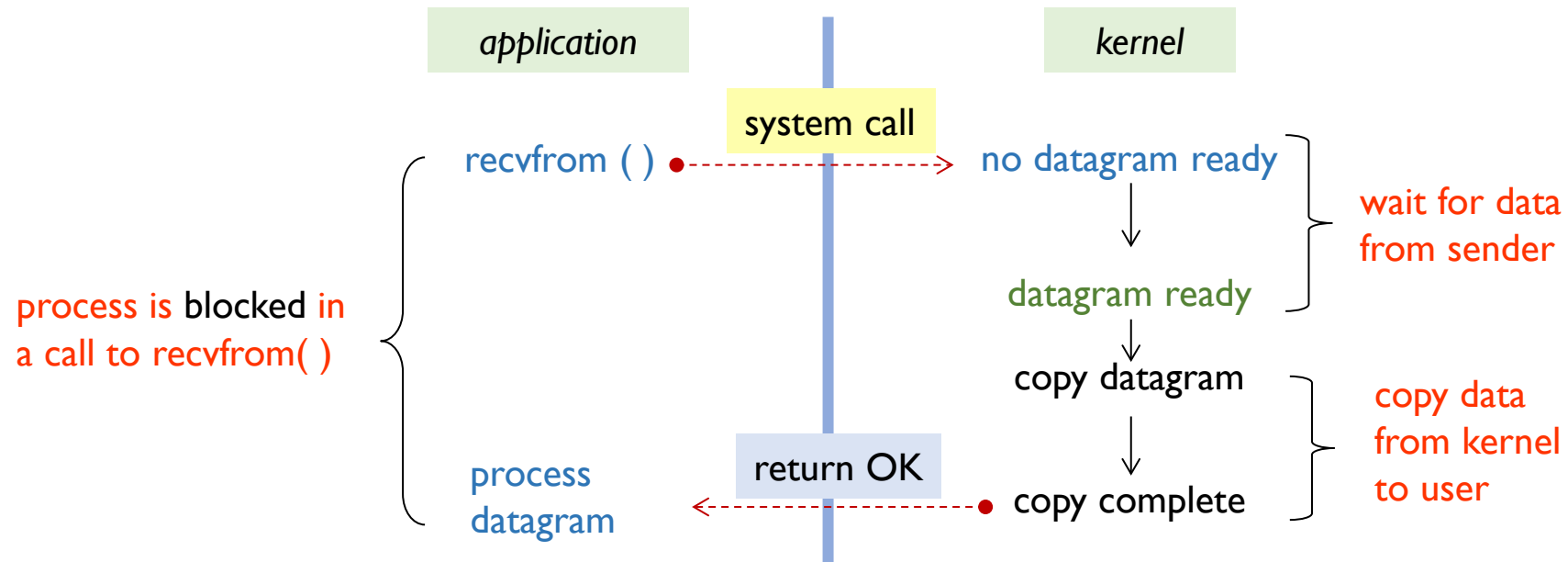



SOCKET TIMEOUT

Timeout

❖ Blocking I/O

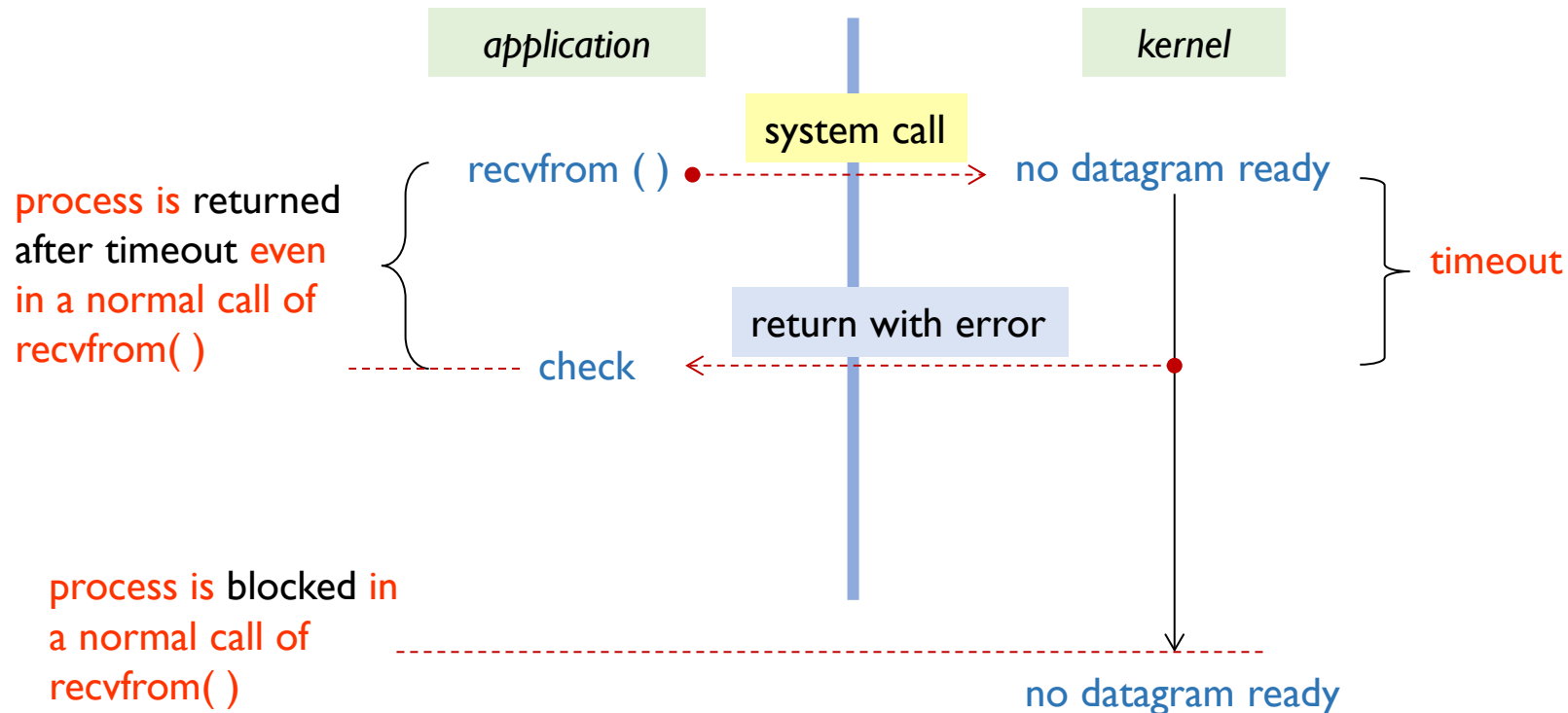
- when a system call is called on a socket,
 - the called process waits until the kernel completes the request
 - `listen( )`, `connect( )`, `accept( )`, `recv( )`, `send( )`, `read( )`, `write( )`, `recvfrom( )`, `sendto( )`, `close( )`, etc.



Timeout

❖ Timeout

- a specified period of time (timeout) for a blocked system call to be returned when there is no response during the time



SO_RCVTIMEO and SO_SNDTIMEO

❖ SO_RCVTIMEO (or SO_SNDTIMEO)

- It defines the timeout (in milliseconds) for blocking receive (or send) calls
- when timeout
 - if data has been received (or sent)
 - the return value will be the amount of data transferred;
 - if no data has been transferred
 - -1 is returned with errno set

```
s = socket (AF_INET, SOCK_DGRAM, 0);

int tout = 1000;                // 1000 milisec. (or 1 sec.)

setsockopt (s, SOL_SOCKET, SO_RCVTIMEO, (char *)&tout, sizeof(t));

setsockopt (s, SOL_SOCKET, SO_SNDTIMEO, (char *)&tout, sizeof(t));
```

SO_RCVTIMEO

❖ example 1

- timeout value is fixed

```
...
s = socket (PF_INET, SOCK_DGRAM, 0);
...

int tout = 1000;                                     //1000=1 sec
setsockopt (s, SOL_SOCKET, SO_RCVTIMEO, (char *) &tout, sizeof(tout));

...
while (1) {
    sendto (s, ...) ;

    recvMsgSize = recvfrom (s, ...);

    if (recvMsgSize == SOCKET_ERROR) {
        if ( WSAGetLastError() == WSAETIMEDOUT ) {    // timeout
            printf("receive timeout:");
        }
    }
}
```



SO_RCVTIMEO

- ❖ example 2 (binary exponential backoff) (1/2)
 - algorithm

```
#define max_retransmission_count 16

initially, timeout = 1 sec (for example)
timeout_count = 0;
if ( timeout event occurs ) {
    timeout_count++;
    if ( timeout_count <= max_retransmission_count ) {
        timeout = timeout_count * timeout;
        retransmission with new timeout;
    }
}
```

SO_RCVTIMEO

❖ example 2 (binary exponential backoff) (2/2)

```
...
s = socket (PF_INET, SOCK_DGRAM, 0);
...
while (1) {
    sendto (s, ...) ;    // send a message

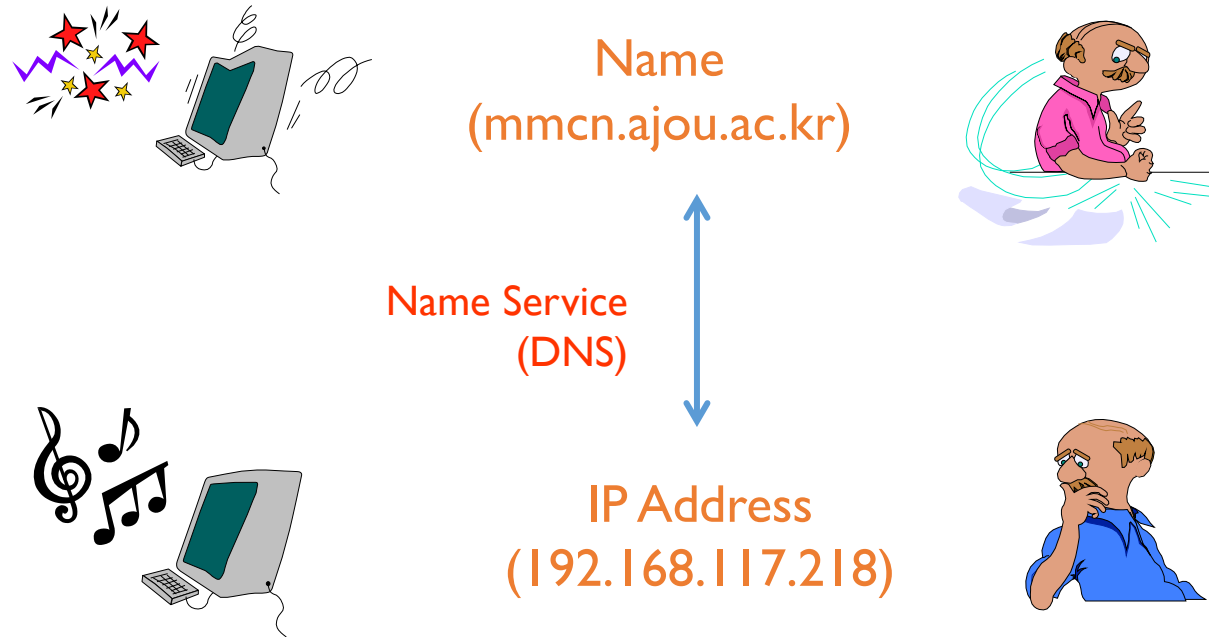
    int tout = 1000;    // initial timeout = 1 sec
    setsockopt (s, SOL_SOCKET, SO_RCVTIMEO, (char *) &tout, sizeof(tout));
    while ( 1 ) {
        recvMsgSize = recvfrom (s, ...);    // blocked call
        if (recvMsgSize == SOCKET_ERROR) {
            if ( WSAGetLastError() == WSAETIMEDOUT ) {    // occurs timeout event
                tout *= 2;    // doubles timeout value
                setsockopt (s, SOL_SOCKET, SO_RCVTIMEO, (char *) &tout, sizeof(tout));
                sendto (s, ...)    // retransmission
                continue;
            }
        }
        break;    // when successful, send a next message
    }
}
```



OTHER SOCKET FUNCTIONS

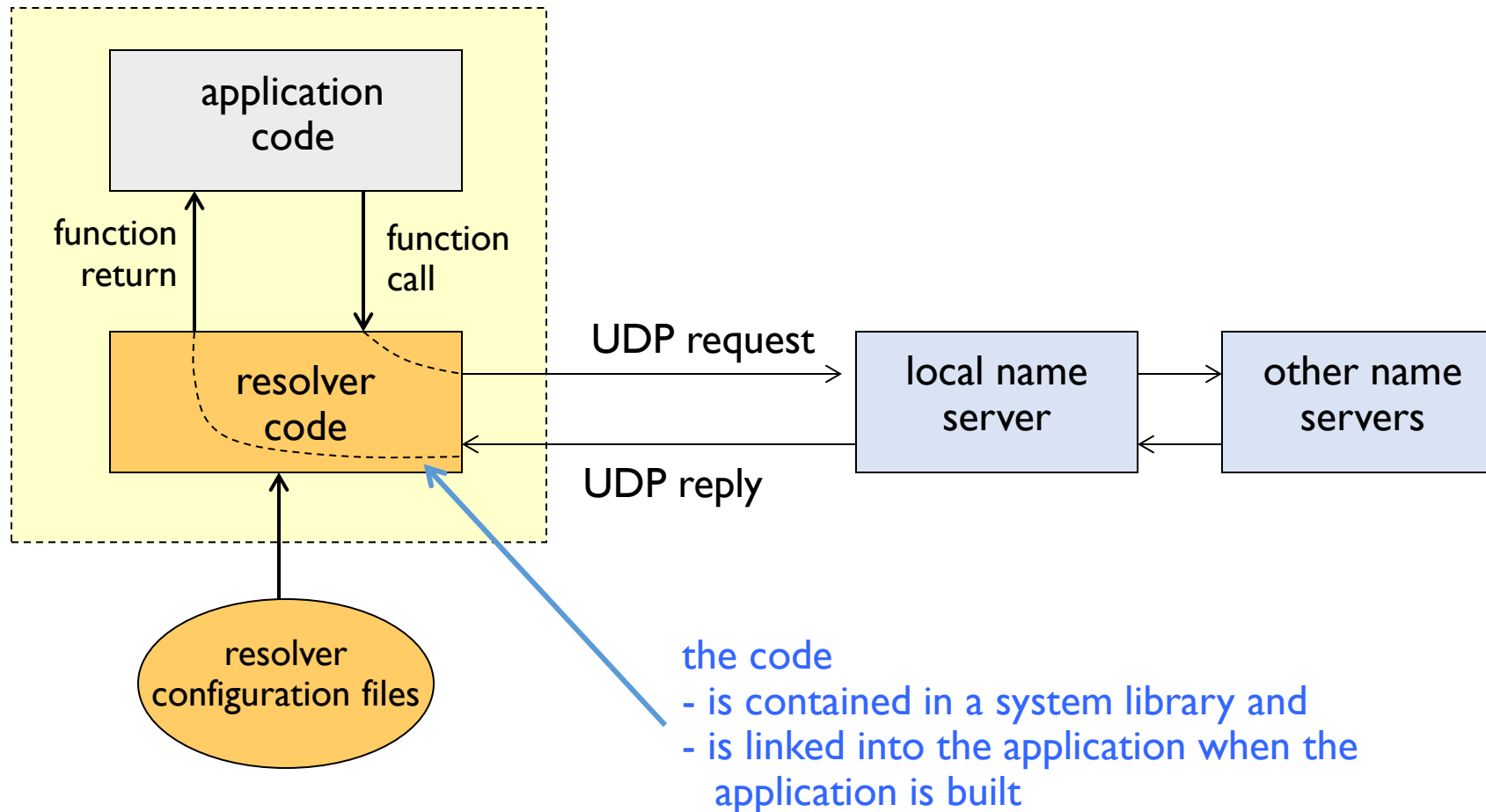
Name/Address Conversion

- ❖ So far,
 - all the examples have used *numeric addresses* and *numeric port numbers*



Name/Address Conversion

❖ Name Resolution Procedure





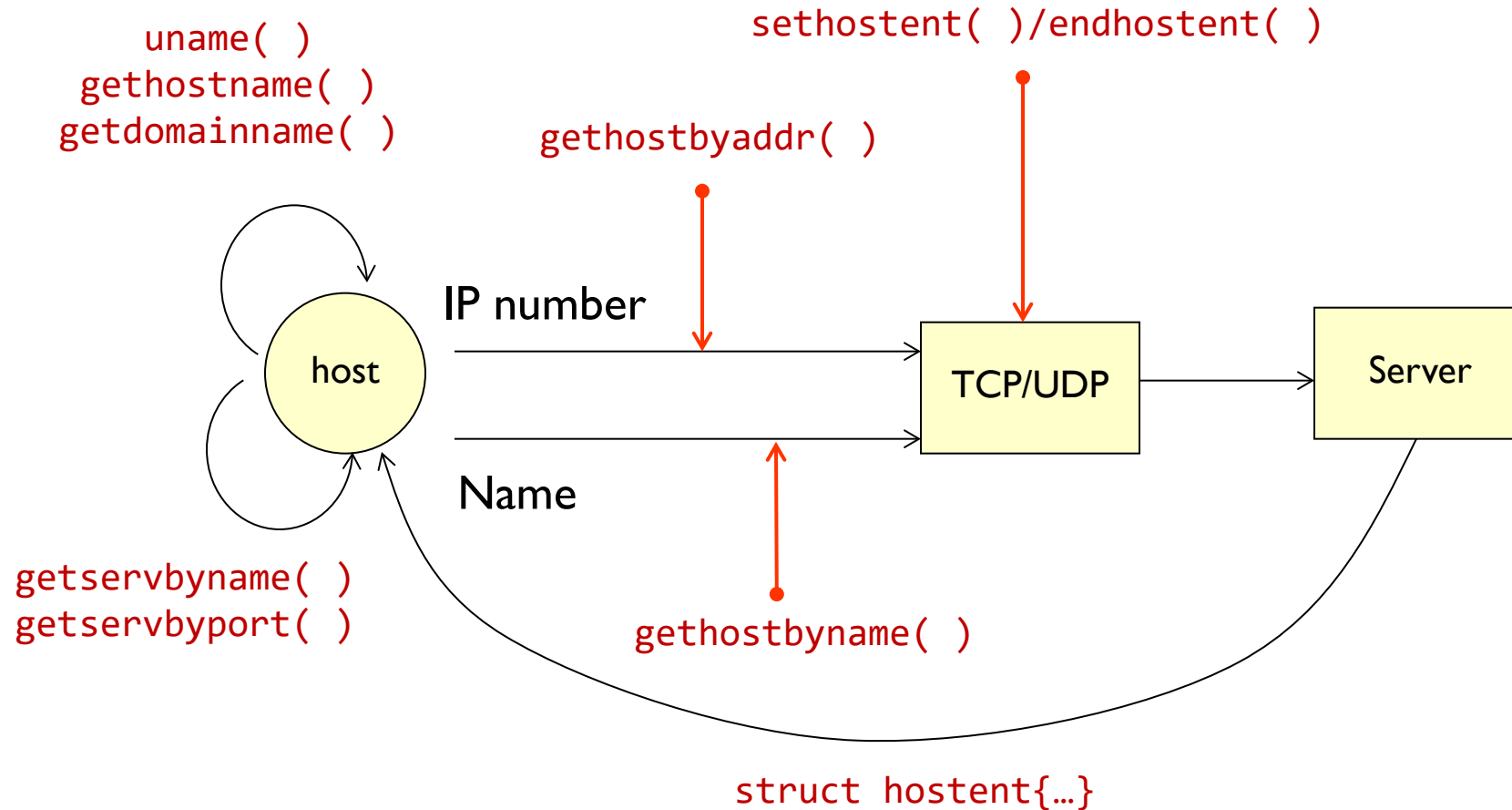
Basic Name Lookup Functions

❖ Functions for name lookups

- to get local host information
 - `uname( )`
 - `gethostname( )`
 - `getdomainname( )`
- to convert between names and IP addresses
 - `gethostbyname( )`
 - `gethostbyaddr( )`
- to convert between service names and port numbers
 - `getservbyname( )`
 - `getservbyport( )`

Basic Name Lookup Functions

❖ Summary



System Information (1/2)

❖ uname() function

- to get name and information about current kernel

```
#include <sys/utsname.h>
int uname ( struct utsname *buf );
```

- return value

- 0 on success, -1 on error, with errno

- structure for returning the system information

```
struct utsname {
    char    sysname [SYS_NMLN];    /* OS name: e.g. Linux*/
    char    nodename [SYS_NMLN];   /* machine's node hostname */
    char    release [SYS_NMLN];    /* OS release: e.g. 2.4.18 */
    char    version [SYS_NMLN];    /* OS ver. date of kernel build*/
    char    machine [SYS_NMLN];    /* hardware type : e.g. i686 */
    char    domainname [SYS_NMLN]; /* NIS/YP domain name */
}
```

System Information (2/2)

❖ uname() - example

```
struct utsname u_name;  
int z = uname (&u_name);  
  
printf (" sysname[] = '%s';\n",    u_name.sysname);  
printf (" nodename[] = '%s';\n",    u_name.nodename);  
printf (" release[] = '%s';\n",    u_name.release);  
printf (" version[] = '%s';\n",    u_name.version);  
printf (" machine[] = '%s';\n",    u_name.machine);  
printf (" domainname[] = '%s';\n", u_name.domainname);
```



```
sysname[] = 'Linux';  
nodename[] = 'ubuntu';  
release[] = '3.16.0-34-generic';  
version[] = '#47~14.04.1-Ubuntu SMP Fri Apr 10 17:49:51 UTC 2015';  
machine[] = 'i686';  
domainname[] = '(none)';
```



Host and Domain Names (1/2)

❖ to access or to change the host-name

```
int  gethostname (char *name, size_t len) ;  
int  sethostname (const char *name, size_t len);
```

❖ to access or to change the NIS domain-name

```
int  getdomainname (char *name, size_t len);  
int  setdomainname (const char *name, size_t len);
```

■ arguments

- **name** : buffer to access or set the name information
- **len** : the maximum size of name (in bytes)

■ return values

- **0** on success, **-1** on error with external variable **errno**

Host and Domain Names (2/2)

❖ example

```
int z = gethostname (buf, sizeof buf);  
printf("host name = '%s'\n", buf);  
  
z = getdomainname (buf, sizeof buf);  
printf("domain name = '%s'\n", buf);
```



```
host name = 'ubuntu'  
domain name = '(none)'
```


Name-to-Address Resolution (1/3)

❖ basic functions

- to get a structure *hostent* for the given host name (name→IP)

```
struct hostent *gethostbyname ( const char *name);
```

• string of the hostname

- to get a structure *hostent* for the given host address (IP→name)

```
struct hostent *gethostbyaddr ( const char *addr,  
                                int len,  
                                int type);
```

return value

- a pointer to the hostent structure on success
- NULL on error with the reason h_errno

• type of the input address(e.g. AF_INET)

• length of the input address (in bytes)

• the input address (sin_addr for AF_INET)

Name-to-Address Resolution (2/3)

❖ struct hostent structure

```
struct hostent {  
    char    *h_name;      /* official name of host */  
    char    **h_aliases; /* alias list */  
    Int     h_addrtype;   /* host address type */  
    Int     h_length;     /* length of address */  
    char    **h_addr_list; /* list of addresses */  
}
```

official hostname \0

alias #1 \0

alias #2 \0

NULL

AF_INET

4 (bytes)

IP addr #1

IP addr #2

IP addr #3

NULL

(network byte ordered)

Name-to-Address Resolution (3/3)

❖ example code

```
struct hostent *ptr;

ptr = gethostbyname ("www.google.com");

printf("Host %s :\n", argv[x]);
printf("  Officially:\t%s\n", hp->h_name);
fputs("  Aliases:\t", stdout);

for ( int x2=0; hp->h_aliases[x2]; ++x2 )
    fputs(hp->h_aliases[x2], stdout);
printf("  Type:\t%s\n", hp->h_addrtype == AF_INET ? "AF_INET" : "AF_INET6");

if ( hp->h_addrtype == AF_INET )
    for ( x2=0; hp->h_addr_list[x2]; ++x2 )
        printf("  Address:\t%s\n",
            inet_ntoa( *(struct in_addr *) hp->h_addr_list[x2]));
```

```
Host www.google.com :
  Officially:    www.google.com
  Aliases:
  Type: AF_INET
  Address:      173.194.126.244
  Address:      173.194.126.240
  Address:      173.194.126.241
  Address:      173.194.126.242
  Address:      173.194.126.243
```



getaddrinfo() Function

❖ Protocol-dependent name lookup functions

- For IPv4, gethostbyname()
- For IPv6, gethostbyname2()

❖ getaddrinfo()

- Protocol-independent hostname-to-address translation

```
#include <netdb.h>
int getaddrinfo ( char                *hostname,
                  const char          *servname,
                  const struct addrinfo *hints,
                  const struct addrinfo **res);
```

getaddrinfo() Function

❖ getaddrinfo()

■ arguments

- **hostname** : hostname or an address string (for IPv4 or IPv6)
- **service** : service name or port number
- **hints** : hints concerning the desired return information
- **res** : result

■ struct addrinfo structure

```
struct addrinfo {  
    int             ai_flags;          // AI_PASSIVE, AI_CANONNAME  
    int             ai_family;         // PF_xxx  
    int             ai_socktype;       // SOCK_xxx  
    int             ai_protocol;       // 0 or IPPROTO_xxx for IPv4 and IPv6  
    size_t          ai_addrlen;        // length of ai_addr  
    char            *ai_canonname;     // canonical name for hostname  
    struct sockaddr *ai_addr;          // binary address  
    struct addrinfo *ai_next;          // next structure in linked list  
};
```



getaddrinfo() Function

❖ example

```
struct addrinfo hints, *res;  
  
memset (&hints, 0, sizeof hints);  
hints.ai_family = AF_INET;  
  
getaddrinfo ("mmcn.ajou.ac.kr", "http", &hints, &res);
```

Questions ?

MR-IoT/AI Convergence

Future Internet & 5G/6G

Intelligent Networking with AI

MMcN



Mobile **Multimedia Convergence** Network Lab.

Homepage: <http://mmcn.ajou.ac.kr>

facebook: <http://www.facebook.com/#!/groups/190702010947314>

